week 4

Day 1

- Had our midterm Presentation
- Got some interesting and helpful inputs and feedback

Feedback

- Work with a text-to-speech functionality
- Integrate the song “Scatman” maybe combine it with morse code
- Increase the number of variables in the code
- Incorporate color combinations directly into the live coding

Day 2

- Excursion to basel

Day 3

- We received some interesting input from Stefanie
- After that, we read various texts on the topic in small groups
- I read the text “Female Electronics or Another History of Electronic Music”
- Completed the related assignments
- Discussed our answers in the group

Day 3

- Selfstudy
- We continued working on our codes
- Initial implementations using Braille and Morse code
- Coded various implementations with this theme
- We thought about what our concept might look like and how we could turn our ideas into a cohesive performance

```
// {"P5LIVE":{"name":"blindensprache","mod":1779376291748}}

let input;
let words = [];
let wordIndex = 0;
let letterIndex = 0;

let t = 0;
```

```
let bgColor;
let started = false;

let synth;
let lastChar = "";

let braille = {
  a: [1], b: [1, 3], c: [1, 2], d: [1, 2, 4], e: [1, 4],
  f: [1, 2, 3], g: [1, 2, 3, 4], h: [1, 3, 4], i: [2, 3], j: [2, 3, 4],
  k: [1, 5], l: [1, 3, 5], m: [1, 2, 5], n: [1, 2, 4, 5], o: [1, 4, 5],
  p: [1, 2, 3, 4, 5], r: [1, 3, 4, 5], s: [2, 3, 5], t: [2, 3, 4, 5],
  u: [1, 5, 6], v: [1, 3, 5, 6], w: [2, 3, 4, 6],
  x: [1, 2, 5, 6], y: [1, 2, 4, 5, 6], z: [1, 4, 5, 6]
};

let notes = {
  a: 110, b: 116, c: 123, d: 131, e: 147,
  f: 165, g: 175, h: 196, i: 220, j: 247,
  k: 262, l: 294, m: 330, n: 349, o: 392,
  p: 440, r: 494, s: 523, t: 587,
  u: 659, v: 698, w: 784,
  x: 880, y: 988, z: 1047
};

let anim = [0, 0, 0, 0, 0, 0];

function setup() {
  createCanvas(windowWidth, windowHeight);

  bgColor = color(0, 0, 255);

  // Sound
  synth = new p5.Oscillator('sine');
  synth.start();
  synth.amp(0);

  input = createInput("");
  input.position(20, 20);

  input.input(() => {
    let text = input.value().toLowerCase().trim();

    if (text.length > 0) {
      words = text.split(" ").filter(w => w.length > 0);
      wordIndex = 0;
      letterIndex = 0;
      started = true;
    } else {
      started = false;
    }
```

```
  });
}

function draw() {
  background(bgColor);

  if (!started) {
    fill(255);
    textAlign(CENTER);
    textSize(20);
    text("Text eingeben um zu starten", width / 2, height / 2);
    return;
  }

  // Tempo
  if (millis() - t > 180) {
    letterIndex++;
    t = millis();
  }

  let word = words[wordIndex % words.length] || "";
  let ch = word[letterIndex] || "";

  // SOUND (nur wenn neuer Buchstabe!)
  if (ch !== lastChar) {
    let freq = notes[ch];

    if (freq) {
      synth.freq(freq);
      synth.amp(0.35, 0.05);

      setTimeout(() => {
        synth.amp(0, 0.08);
      }, 80);
    }

    lastChar = ch;
  }

  // Wortwechsel
  if (letterIndex >= word.length) {
    wordIndex++;
    letterIndex = 0;

    bgColor = color(
      random(50, 255),
      random(50, 255),
      random(50, 255)
    );
  }
```

```
  let dots = braille[ch] || [];

  for (let n = 1; n <= 6; n++) {
    let target = dots.includes(n) ? 1 : 0;
    anim[n - 1] = lerp(anim[n - 1], target, 0.25);
  }

  drawBraille();

  fill(255);
  textAlign(CENTER);
  textSize(28);
  text(word, width / 2, height / 2 + 200);
}

function drawBraille() {
  let cx = width / 2;
  let cy = height / 2;

  let s = min(width, height) * 0.13;

  let pos = [
    [-0.6, -0.8], [0.6, -0.8],
    [-0.6, 0], [0.6, 0],
    [-0.6, 0.8], [0.6, 0.8]
  ];

  for (let n = 1; n <= 6; n++) {
    let x = cx + pos[n - 1][0] * s;
    let y = cy + pos[n - 1][1] * s;

    let a = anim[n - 1];

    fill(255, 120);
    noStroke();
    circle(x, y, s * 0.2);

    fill(255);
    circle(x, y, s * 0.2 + a * (s * 0.4));
  }
}
```

P5L_blindensprache_20260521151131.png

```
// {"P5LIVE":{"name":"Morsecode random","mod":1779378036408}}

/*
```

```
    _HY5_p5_hydra // cc teddavis.org 2024
    pass p5 into hydra
    docs: https://github.com/ffd8/hy5


let libs = ['https://unpkg.com/hydra-synth', 'includes/libs/hydra-
synth.js', 'https://cdn.jsdelivr.net/gh/ffd8/hy5@main/hy5.js',
'includes/libs/hy5.js']

// sandbox - start
H.pixelDensity(2) // 2x = retina, set <= 1 if laggy

s0.initP5() // send p5 to hydra
P5.toggle(0) // optionally hide p5
src(s0)
    .modulate(noize(2))
    .out()

// sandbox - end
*/
let typedText = "";
let morseItems = [];
let synth;
const morseSize = 2;

const morse = {
  a: ".-", b: "-...", c: "-.-.", d: "-..",
  e: ".", f: "..-.", g: "--.", h: "....",
  i: "..", j: ".---", k: "-.-", l: ".-..",
  m: "--", n: "-.", o: "---", p: ".--.",
  q: "--.-", r: ".-.", s: "...", t: "-",
  u: "..-", v: "...-", w: ".--",
  x: "-..-", y: "-.--", z: "--.."
};

// Loop-State
let lastTypedTime = 0;
let loopIndex = 0;
let loopTimeout = null;
let isLooping = false;
const IDLE_DELAY = 2000; // ms bis Loop startet

function setup() {
  createCanvas(windowWidth, windowHeight);
  synth = new p5.Oscillator('sine');
  synth.start();
  synth.amp(0);
  textFont('monospace');
  lastTypedTime = millis();
}
```

```
function draw() {
  background(20, 30, 60);

  for (let item of morseItems) {
    drawMorse(item);
  }

  fill(255);
  textAlign(CENTER);
  textSize(40);
  text(typedText, width / 2, height - 80);

  // Loop starten wenn idle und Zeichen vorhanden
  if (!isLooping && morseItems.length > 0 && millis() - lastTypedTime >
IDLE_DELAY) {
    isLooping = true;
    loopIndex = 0;
    playNextLoopItem();
  }
}

function keyTyped() {
  let k = key.toLowerCase();

  if (morse[k]) {
    typedText += k;
    morseItems.push({
      code: morse[k],
      x: random(150, width - 150),
      y: random(150, height - 150)
    });
    playMorse(morse[k]);
  }

  if (k === " ") {
    typedText += " ";
  }

  // Loop unterbrechen bei Eingabe
  stopLoop();
  lastTypedTime = millis();
}

function keyPressed() {
  if (keyCode === BACKSPACE) {
    typedText = typedText.slice(0, -1);
    if (morseItems.length > 0) {
      morseItems.pop();
    }
```

```
  }
  stopLoop();
  lastTypedTime = millis();
}

function drawMorse(item) {
  let spacing = 30 * morseSize;
  for (let i = 0; i < item.code.length; i++) {
    let sym = item.code[i];
    let px = item.x + (i - item.code.length / 2) * spacing;
    let py = item.y;
    fill(255);
    noStroke();
    textAlign(CENTER, CENTER);
    textSize(40 * morseSize);
    if (sym === ".") text(".", px, py);
    if (sym === "-") text("–", px, py);
  }
}

function playMorse(code) {
  let time = 0;
  for (let sym of code) {
    let duration = sym === "." ? 120 : 350;
    setTimeout(() => {
      synth.freq(400);
      synth.amp(0.35, 0.02);
      setTimeout(() => synth.amp(0, 0.05), duration);
    }, time);
    time += duration + 120;
  }
  // Gesamtdauer zurückgeben (für Loop-Timing)
  return time;
}

function getMorseDuration(code) {
  let time = 0;
  for (let sym of code) {
    let duration = sym === "." ? 120 : 350;
    time += duration + 120;
  }
  return time;
}

function playNextLoopItem() {
  if (!isLooping || morseItems.length === 0) return;

  let item = morseItems[loopIndex % morseItems.length];
  let duration = getMorseDuration(item.code);
```

```
  playMorse(item.code);

  loopIndex++;

  // Pause zwischen Zeichen: 400ms, dann nächstes
  loopTimeout = setTimeout(() => {
    playNextLoopItem();
  }, duration + 400);
}

function stopLoop() {
  isLooping = false;
  if (loopTimeout !== null) {
    clearTimeout(loopTimeout);
    loopTimeout = null;
  }
  synth.amp(0, 0.05);
}
```

P5L_Morsecode random_20260521154036.png

```
// {"P5LIVE":{"name":"Morsecode Wörter","mod":1779377924309}}

// === VARIABLEN ===
let input, words = [], wordIndex = 0, letterIndex = 0, symbolIndex = 0;
let t = 0, bgColor, started = false, synth, currentSymbols = "";
let currentWordMorse = "";

const morseSize = 2; // Größe der Morse-Symbole
const morse = {
  a:".-", b:"-...", c:"-.-.", d:"-..", e:".", f:"..-.", g:"--.", h:"....",
  i:"..", j:".---", k:"-.-", l:".-..", m:"--", n:"-.", o:"---", p:".--.",
  q:"--.-", r:".-.", s:"...", t:"-", u:"..-", v:"...-", w:".--",
  x:"-..-", y:"-.--", z:"--.."
};

function setup() {
  createCanvas(windowWidth, windowHeight);
  bgColor = color(20, 80, 180);
  synth = new p5.Oscillator('sine');
  synth.start();
  synth.amp(0);
  input = createInput("");
  input.position(20, 20);
  input.input(() => {
    const text = input.value().toLowerCase().trim();
    words = text.split(" ").filter(w => w.length > 0);
    started = words.length > 0;
    wordIndex = letterIndex = symbolIndex = 0;
```

```
    if (started) updateWordMorse();
  });
}

function draw() {
  background(bgColor);
  fill(255); textAlign(CENTER);
  if (!started) {
    textSize(20);
    text("Text eingeben um zu starten", width / 2, height / 2);
    return;
  }
  if (millis() - t > 800) { stepMorse(); t = millis(); }
  drawMorse();
  textSize(150);
  text(words[wordIndex % words.length] || "", width / 2, height / 2 +
200);
}

function updateWordMorse() {
  const word = words[wordIndex % words.length] || "";
  currentWordMorse = word
    .split("")
    .map(ch => morse[ch] || "")
    .join(" ");
}

function stepMorse() {
  const word = words[wordIndex % words.length] || "";
  const code = morse[word[letterIndex]] || "";
  currentSymbols = code;

  if (++symbolIndex >= code.length) {
    symbolIndex = 0;
    if (++letterIndex >= word.length) {
      wordIndex++;
      letterIndex = 0;
      bgColor = color(random(50, 255), random(50, 255), random(50, 255));
      updateWordMorse();
    }
  }
  triggerSound(code[symbolIndex]);
}

function triggerSound(sym) {
  if (!sym) return;
  synth.freq(400);
  synth.amp(0.35, 0.02);
  setTimeout(() => synth.amp(0, 0.05), sym === "." ? 80 : 240);
}
```

```
function drawMorse() {
  const symbols = currentWordMorse.replace(/ /g, ""); // Leerzeichen
entfernen
  const spacing = 40 * morseSize;   // Abstand zwischen Symbolen
  const margin = 60;                // Randabstand links und rechts
  const usableWidth = width - margin * 2; // verfügbare Breite
  const symbolsPerRow = floor(usableWidth / spacing); // wie viele Symbole
pro Zeile passen

  // Symbole in Zeilen aufteilen
  // z.B. 20 Symbole, 8 pro Zeile → [[0-7], [8-15], [16-19]]
  const rows = [];
  for (let i = 0; i < symbols.length; i += symbolsPerRow) {
    rows.push(symbols.slice(i, i + symbolsPerRow));
  }

  const rowHeight = 50 * morseSize; // vertikaler Abstand zwischen Zeilen
  const totalHeight = rows.length * rowHeight; // Gesamthöhe aller Zeilen
  const startY = height / 2 - totalHeight / 2; // Startpunkt so dass alles
vertikal zentriert ist

  // jede Zeile einzeln zeichnen
  for (let r = 0; r < rows.length; r++) {
    const row = rows[r];
    const py = startY + r * rowHeight; // y-Position dieser Zeile

    for (let i = 0; i < row.length; i++) {
      const sym = row[i];
      // x-Position: Zeile zentriert auf dem Screen
      const px = width / 2 + (i - row.length / 2) * spacing;

      fill(255); noStroke();
      if (sym === ".") circle(px, py, 12 * morseSize);
      if (sym === "-") rect(px - 15 * morseSize, py - 6 * morseSize, 30 *
morseSize, 12 * morseSize, 4 * morseSize);
    }
  }
}
```

P5L_Morsecode Wörter_20260521153844.png

```
// {"P5LIVE":{"name":"Morsezeichen 1","mod":1779376651413}}

let input;
let words = [];
let wordIndex = 0;
let letterIndex = 0;
let symbolIndex = 0;
```

```
let t = 0;
let bgColor;
let started = false;

let synth;
let lastSymbol = "";

let morseSize = 4;

let morse = {
  a: ".-",    b: "-...",  c: "-.-.",  d: "-..",   e: ".",
  f: "..-.",  g: "--.",   h: "....",  i: "..",    j: ".---",
  k: "-.-",   l: ".-..",  m: "--",    n: "-.",    o: "---",
  p: ".--.",  q: "--.-",  r: ".-.",   s: "...",   t: "-",
  u: "..-",   v: "...-",  w: ".--",   x: "-..-",  y: "-.--",
  z: "--.."
};

let currentSymbols = "";

function setup() {
  createCanvas(windowWidth, windowHeight);

  bgColor = color(20, 80, 180);

  synth = new p5.Oscillator('sine');
  synth.start();
  synth.amp(0);

  input = createInput("");
  input.position(20, 20);

  input.input(() => {
    let text = input.value().toLowerCase().trim();

    if (text.length > 0) {
      words = text.split(" ").filter(w => w.length > 0);
      wordIndex = 0;
      letterIndex = 0;
      symbolIndex = 0;
      started = true;
    } else {
      started = false;
    }
  });
}

function draw() {
  background(bgColor);
```

```
  if (!started) {
    fill(255);
    textAlign(CENTER);
    textSize(20);
    text("Text eingeben um zu starten", width / 2, height / 2);
    return;
  }

  if (millis() - t > 200) {
    stepMorse();
    t = millis();
  }

  drawMorse();

  let word = words[wordIndex % words.length] || "";
  fill(255);
  textAlign(CENTER);
  textSize(150);
  text(word, width / 2, height / 2 + 200);
}

function stepMorse() {
  let word = words[wordIndex % words.length] || "";
  let ch = word[letterIndex];

  if (!ch) return;

  let code = morse[ch] || "";
  currentSymbols = code;

  symbolIndex++;

  if (symbolIndex >= code.length) {
    symbolIndex = 0;
    letterIndex++;

    if (letterIndex >= word.length) {
      wordIndex++;
      letterIndex = 0;

      bgColor = color(
        random(50, 255),
        random(50, 255),
        random(50, 255)
      );
    }
  }
```

```
  triggerSound(code[symbolIndex]);
}

function triggerSound(sym) {
  if (!sym) return;

  // immer gleiche Tonhöhe
  synth.freq(400);

  // Punkt = kurz
  if (sym === ".") {
    synth.amp(0.35, 0.02);

    setTimeout(() => {
      synth.amp(0, 0.05);
    }, 80);
  }

  // Strich = lang
  if (sym === "-") {
    synth.amp(0.35, 0.02);

    setTimeout(() => {
      synth.amp(0, 0.05);
    }, 240);
  }
}

function drawMorse() {
  let x = width / 2 + 70;
  let y = height / 2;

  let spacing = 40 * morseSize;

  for (let i = 0; i < currentSymbols.length; i++) {
    let sym = currentSymbols[i];

    let px = x + (i - currentSymbols.length / 2) * spacing;


    if (sym === ".") {
      fill(255);
      noStroke();
      circle(px, y, 12 * morseSize);
    }


    if (sym === "-") {
      fill(255);
      noStroke();
```

```
      rect(
        px - 15 * morseSize,
        y - 6 * morseSize,
        30 * morseSize,
        12 * morseSize,
        4 * morseSize
      );
    }
  }
}
```

P5L_Morsezeichen 1_20260521151731.png

```
// {"P5LIVE":{"name":"Morsezeichen 2","mod":1779376874176}}

// input = Textfeld
// words = Wörter-Array
//wordIndex, letterIndex, symbolIndex = wo wir gerade sind
let input, words = [],
    wordIndex = 0,
    letterIndex = 0,
    symbolIndex = 0;
// t = Zeit in Millisekunden
//synth = Ton
// currentSymbols = aktueller Morsecode
let t = 0,
    bgColor, started = false,
    synth, currentSymbols = "";

// Größe der Morse-Symbole
const morseSize = 4;

// Übersetzungstabelle
const morse = { a: ".-",b: "-...",c: "-.-.",d: "-..",
    e: ".",f: "..-.",g: "--.",h: "....",i: "..",j: ".---",
    k: "-.-",l: ".-..",m: "--",n: "-.",o: "---",p: ".--.",
    q: "--.-",r: ".-.",s: "...",t: "-",u: "..-",v: "...-",
    w: ".--",x: "-..-",y: "-.--",z: "--.."
};

//wird einmal beim Start ausgeführt
function setup() {
    // Zeichenfläche und Start-Hintergrundfarbe erstellen
    createCanvas(windowWidth, windowHeight);
    bgColor = color(255, 0, 127);

    // Ton erstellen mit Sinuswelle und stumm starten
    synth = new p5.Oscillator('sine');
    synth.start();
```

```
    // Lautstärke
    synth.amp(0);

    // Texteingabefeld erstellen
    input = createInput("");
    input.position(20, 20);

    // startet wenn man etwas in input tippt
    input.input(() => {
        const text = input.value().toLowerCase().trim(); // In
Kleinbuchstaben umwandeln und kein Leerzeichen
        //.split = Text in Wörter aufteilen -> "hallo welt" = ["hallo",
"welt"]
        //.filter = nur Wörter (w) die länger als 0 sind
        words = text.split(" ").filter(w => w.length > 0);
        started = words.length > 0; // Animation nur starten wenn Wörter
vorhanden
        wordIndex = letterIndex = symbolIndex = 0; // Alle Zähler
zurücksetzen
    });
}

function draw() {
    background(bgColor); // Hintergrund wird Farbe von bgColor
    fill(255);
    textAlign(CENTER);

    // Wenn nichts eingegeben = "Text eingeben um zu starten" anzeigen
    if(!started) {
        textSize(20);
        text("Text eingeben um zu starten", width / 2, height / 2);
        return;
    }

    // millis() Zeit in Millisekunden, die seit Programmstart vergangen
ist
    //Wenn 200ms vergangen = einen Schritt im Morsecode weitergehen
    if(millis() - t > 400) {
        stepMorse();
    //dann Zeitmessung zurücksetzen
        t = millis();
    }

    drawMorse();
    // Aktuelles Wort anzeigen (% = Loop)
    textSize(150)
    text(words[wordIndex % words.length] || "", width / 2, height / 6 +
200);
}
```

```
//schaltet zum nächsten Morse-Symbol weiter ===
function stepMorse() {
     // aktuelles Wort holen
     const word = words[wordIndex % words.length] || "";
     // Morsecode des aktuellen Buchstabens in Tabelle nachschauen
     const code = morse[word[letterIndex]] || "";
// für drawMorse() aktueller Morsecode merken
     currentSymbols = code;

     // symbolIndex erhöhen; wenn wir am Ende des Codes sind → nächster
Buchstabe
     //z.B. "h" = "...." (4 Symbole = code.length = 4)
     // deshalb ++symbolIndex = 4  →  4 >= 4? Ja → symbolIndex = 0,
nächster Buchstabe
     if(++symbolIndex >= code.length) {
     //Index zurück auf 0, damit bei nächsten Buchstaben wieder bei ersten
Symbol anfängt
          symbolIndex = 0;

          // letterIndex erhöhen; wenn wir am Ende des Wortes sind ->
nächstes Wort
          //z.B. word = "hi"  → word.length = 2 / ++letterIndex = 2 -
>nächstes Wort
          if(++letterIndex >= word.length) {
               wordIndex++;
               letterIndex = 0;
          // Zufällige neue Hintergrundfarbe beim Wortwechsel
               bgColor = color(random(50, 255), random(50, 255), random(50,
255));
          }
     }
// Ton für das aktuelle Symbol abspielen
     triggerSound(code[symbolIndex]);
}

function triggerSound(sym) {
     if(!sym) return; // kein Symbol = kein Ton

     synth.freq(400); // Tonhöhe
     synth.amp(0.35, 0.02); // Erste Zahl = Lautstärke, wie schnell so
laut = zweite Zahl

//sym === "."  ->  true  -> dann 80ms   (kurz)
//sym === "."  -> false -> dann 240ms  (lang)
     setTimeout(() => synth.amp(0, 0.05), sym === "." ? 80 : 240);
}

function drawMorse() {
     const spacing = 40 * morseSize; // Abstand zwischen den Symbolen
//Für jedes Symbol in currentSymbols einmal drawMorse aufgerufen
```

```
//jedes Zeichen einzeln an der richtigen Position gezeichnet
//nach jedem Durchlauf i um 1 erhöhen
     for(let i = 0; i < currentSymbols.length; i++) {
          const sym = currentSymbols[i];

          // Position und Farbe
          const px = width / 2 + 70 + (i - currentSymbols.length / 2) *
spacing;
          const py = height / 2 + 100;
          fill(255);
          noStroke();
// Symbole für . und - zeichenen
          if(sym === ".") circle(px, py, 12 * morseSize);
// rect(x, y, breite, höhe, rundung)
          if(sym === "-") rect(
               px - 15 * morseSize, py - 6 * morseSize,
               30 * morseSize, 12 * morseSize,
               4 * morseSize
          );
     }
}
```

P5L_Morsezeichen 2_20260521152114.png

Day 5

- Yann showed us how to create 3D shapes in p5live
- After that, we made them audio-reactive
- We looked at the different variations using tools like fftEase
- For me, the input was very helpful in refining my own code

```
// {"P5LIVE":{"name":"Audio Control 3D Shapes","mod":1780313774659}}

/*
     _audio_analysis // cc teddavis.org 2019-24

     revamp of P5LIVE's Audio snippet, now built-in the background!
     add snippet to other sketches: CTRL + SHIFT + A
*/

function setup() {
     createCanvas(windowWidth, windowHeight, WEBGL)

     // audio stuff now behind the scenes, 'true' makes class vars global
     setupAudio(true) // if empty, use 'a5.' before audio vars below
     a5.ease = .275 // customize ease speed (kleiner wert = smooth)
}
```

```
function draw() {
     /* audio vars: amp, ampL, ampR, ampEase, fft, fftEase, waveform,
waveformEase */
     updateAudio()
     lights()
     ambientLight(100)
     background(20, 50, 0)
     fill(255, 115, 0)
     noStroke()
     // orange
     //stroke(255, 115, 0)
     sph(0, fftEase[0] * .72, -50, 200 +fftEase[100])

     // pink
     fill(255, 0, 127)
     noStroke()
     sph(300, -20, 30, 100 +fftEase[20])

     // grün
     fill(0, 255, 230)
     noStroke()
     trs(-50, -200, 100, 50 +fftEase[60], 10)
}

function sph(x, y, z, size) {
     push()
     translate(x, y, z)
     sphere(size)
     pop()
}

function trs(x, y, z, size, rSpeed) {
     push()
     translate(x, y, z)
     rotateY(rSpeed * frameCount * 0.01)

     torus(size)
     pop()
}




     // /* fftEase */
     // for(let i = 0; i < fftEase.length; i++) {
     //   let freq = fftEase[i]; // (0, 255)
     //   let x = map(i, 0, fftEase.length, 0, width)
     //   let w = width / fftEase.length
     //   rect(x, height * .805, w, freq)
     // }
```

```
/*
P5LIVE - Audio
If using outside P5LIVE, include p5live-audio.js
https://cdn.jsdelivr.net/gh/ffd8/P5LIVE/includes/utils/p5live-audio.js
*/
```

P5L_Audio Control 3D Shapes_20260601113614.png

```
// {"P5LIVE":{"name":"Audio Control 3D 2","mod":1780313781402}}

/*
    _audio_analysis // cc teddavis.org 2019-24

    revamp of P5LIVE's Audio snippet, now built-in the background!
    add snippet to other sketches: CTRL + SHIFT + A
*/

function setup() {
    createCanvas(windowWidth, windowHeight, WEBGL)

    // audio stuff now behind the scenes, 'true' makes class vars global
    setupAudio(true) // if empty, use 'a5.' before audio vars below
    a5.ease = .275 // customize ease speed (kleiner wert = smooth)
}

function draw() {
    background(0)
    /* audio vars: amp, ampL, ampR, ampEase, fft, fftEase, waveform,
waveformEase */
    updateAudio()

    orbitControl()



    // reddish light; first 3 values are colour, rest is point (x,y,z)
where it originates
    stroke(229, 138, 255)
    fill(138, 255, 151)

    //gives objects a 3d look by adding lights and shadows
    lights()
    directionalLight(0, 0, 255, -10, 10, 0)

    let number = 6
    let index = 0

    for(let x = 0; x < number; x++) {
```

```
          let posX = map(x, 0, number - 1, -width / 4, width / 4)
          for(let y = 0; y < number; y++) {
               let posY = map(y, 0, number - 1, -width / 4, width / 4)
               for(let z = 0; z < number; z++) {
                    let posZ = map(z, 0, number - 1, -width / 4, width / 4)
                    if(index % 2 === 0){
                    cube(posX, posY, posZ, 40 +
(fftEase[index%fftEase.length]*0.25), 1)
                    }else{
                         sph(posY,posY,posZ, 20 +
(fftEase[index%fftEase.length]*0.25), 1)
                    }
                    index++
               }
          }
     }

}

function sph(x, y, z, size, rSpeed) {
     push()
     translate(x, y, z)
     rotateY(frameCount * 0.01 * rSpeed)
     sphere(size)
     pop()
}

function cube(x, y, z, size, rSpeed) {
     push()
     translate(x, y, z)
     rotateZ(frameCount * 0.01 * rSpeed)
     rotateY(frameCount * 0.01 * rSpeed)
     box(size)
     pop()
}

function trs(x, y, z, size, rSpeed) {
     push()
     translate(x, y, z)
     rotateY(frameCount * 0.01 * rSpeed)
     torus(size, 20,10,6)
     pop()
}

// /* fftEase */
// for(let i = 0; i < fftEase.length; i++) {
//   let freq = fftEase[i]; // (0, 255)
//   let x = map(i, 0, fftEase.length, 0, width)
//   let w = width / fftEase.length
//   rect(x, height * .805, w, freq)
```

```
// }

/*
P5LIVE - Audio
If using outside P5LIVE, include p5live-audio.js
https://cdn.jsdelivr.net/gh/ffd8/P5LIVE/includes/utils/p5live-audio.js
*/
```

P5L_Audio Control 3D 2_20260601113621.png